

EE2213 Introduction to Artificial Intelligence

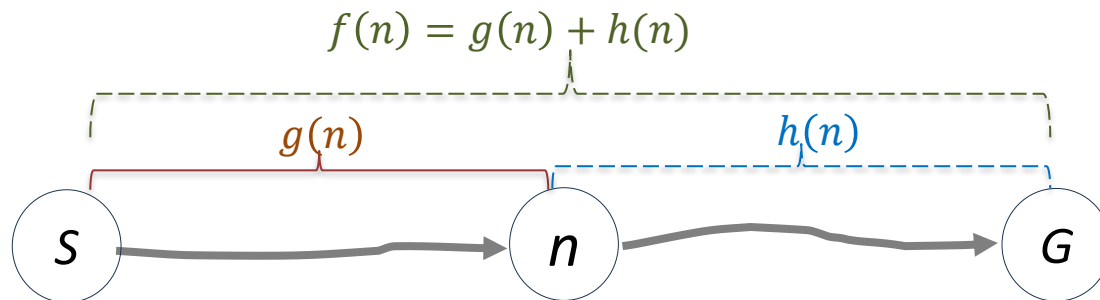
Tutorial 3 (Lectures 3 & 4)

Dr Shaojing Fan
fanshaojing@nus.edu.sg

A*: Where *heuristics* meet *reality*

It evaluates nodes by combining $g(n)$, the actual cost to reach the node, and $h(n)$, the estimated cost to get from the node to the goal.

$f(n) = g(n) + h(n)$  $f(n)$: *Estimated total cost* of the *cheapest path* from the start to the goal via node n



$g(n)$: Actual cost of the path from the start node to the current node n . (for start node: $g(n) = 0$)

$h(n)$: Estimated cost of the cheapest path from the current node n to the goal. (for goal node: $h(n) = 0$)

Recap: Admissible heuristics

A heuristic $h(n)$ is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the **true cost** to reach the goal state from n .

Admissible heuristics are **optimistic**: they **never overestimate** the cost to reach the goal, always estimating it as **less than or equal** to the actual cost.

Example:

Driving from City A to B

- Actual road distance: 100 km
- Straight-line (Euclidean) distance as heuristic: 80 km

The heuristic **never overestimates** → **Admissible**



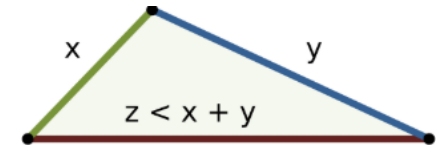
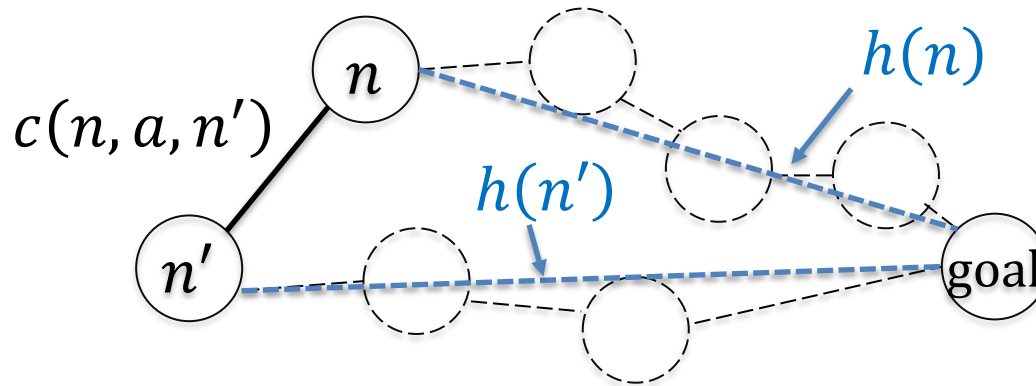
Recap: Consistent heuristics

A heuristic $h(n)$ is **consistent** (or monotonic) if for every node n , every successor n' of n generated by any action a :

$$h(n) \leq c(n, a, n') + h(n')$$

$c(n, a, n')$ is the actual cost of applying action a to go from state n to n'

Consistency is a version of triangle inequality.



Consistency is a **stricter requirement** than admissibility.

Q1. Which of the following search algorithms typically use a priority queue to implement the frontier? Select all that apply.

- A. Breadth-first search
- B. Uniform cost search (i.e., Dijkstra's algorithm)
- C. Greedy best-first search
- D. A* search

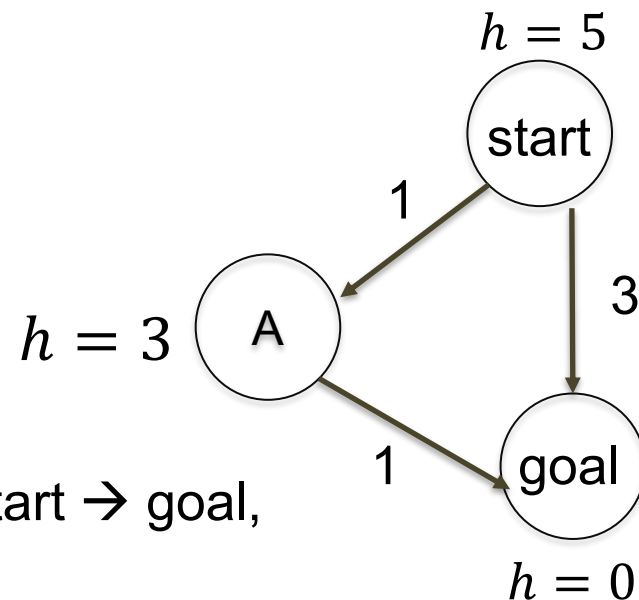
Q1. Which of the following search algorithms typically use a priority queue to implement the frontier? Select all that apply.

- A. Breadth-first search
- B. Uniform cost search (i.e., Dijkstra's algorithm) ✓ prioritize based on $g(n)$
- C. Greedy best-first search ✓ prioritize based on $h(n)$
- D. A* search ✓ prioritize based on $f(n) = g(n) + h(n)$

Q2. Which of the following is true about A* search? Select all that apply.

- A. The heuristic function can overestimate the true cost to the goal without affecting the optimality of A* search.
- B. A* search is generally complete and returns a solution if one exists, provided the search space is finite and all step costs are positive.
- C. Every consistent heuristic is also admissible.
- D. The graph-search of A* is optimal as long as the heuristic is admissible.

Example when heuristic function can overestimate the true cost:



Given $h(A) = 3$, A^* chooses the path $\text{start} \rightarrow \text{goal}$, resulting in a **total cost of 3**

However, this is not the optimal path— A^* fails due to the heuristic being inadmissible ($h(A) > h^*(A)$).

We should design heuristics to be optimistic—that is, they never overestimate the true cost to reach the goal. This property defines **admissibility**.

Recap: A* search properties

Optimality

- **Tree search** version of A* is optimal if the heuristic is *admissible*.
- **Graph search** version of A* is optimal if the heuristic is *consistent*.

Completeness

- A* search is generally complete (provided finite branching factor).

Consistency is a **stricter requirement** than admissibility.

Note: These properties of A* — optimality and completeness — **hold only when all edge costs are non-negative**. If negative edge costs exist, A* may fail to find the optimal path or even a solution at all.

Q2. Which of the following is true about A* search? Select all that apply.

- A. The heuristic function can overestimate the true cost to the goal without affecting the optimality of A* search.
- B. A* search is generally complete and returns a solution if one exists, provided the search space is finite and all step costs are positive. ✓
- C. Every consistent heuristic is also admissible. ✓
- D. The graph-search of A* is optimal as long as the heuristic is admissible.

Q3. Prove the following statement:

If a heuristic $h(n)$ is consistent, then it is also admissible.

Hint: Proof by induction

Q3. Prove if a heuristic $h(n)$ is consistent, then it is also admissible

Admissible heuristic:

For every node n ,

$$h(n) \leq h^*(n),$$

where $h^*(n)$ is the **actual** cost to from n to goal.

To be admissible, **$h(\text{goal}) = 0$**

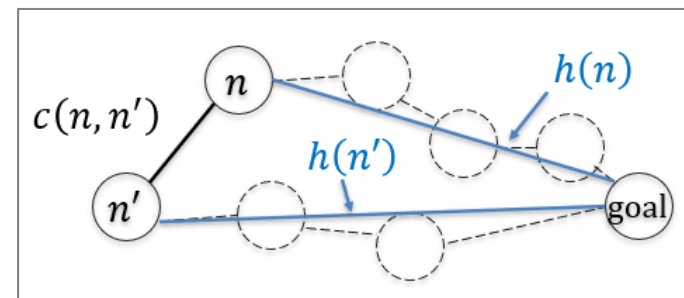
Consistent heuristic:

For every node n and successor n' with step cost $c(n, a, n')$ by any action a :

$$h(n) \leq c(n, n') + h(n')$$

and

$$h(\text{goal}) = 0$$



The notation $c(n, n')$ is a simplified version of $c(n, a, n')$

Q3. Prove if a heuristic $h(n)$ is consistent, then it is also admissible

Proof by induction

1. Show it is true for a **base case** ($n = \text{goal}$)
2. Assume it is true for all nodes at most k steps away from the goal
3. Use step 2 to show for $k + 1$: Prove it holds for nodes at most $k + 1$ steps away from the goal

Induction is a key technique for proving the correctness of search algorithms, especially when reasoning about properties of paths or heuristics across multiple steps in the search process.

Q3. Prove if a heuristic $h(n)$ is consistent, then it is also admissible

Proof by induction

Base case:

For the goal node $n = \text{goal}$,

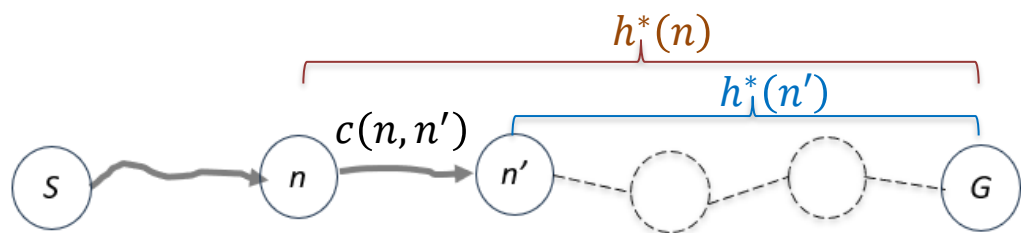
By definition:

$$h(\text{goal}) = 0$$

$$h(\text{goal}) \leq h^*(\text{goal}) \text{ holds}$$

So the heuristic is admissible (doesn't overestimate).

Inductive Hypothesis:



Assume that any node n' that is at most k steps away from the goal along the shortest path satisfies the admissible property:

$$h(n') \leq h^*(n') \quad (\text{i.e., } h \text{ is admissible for those nodes})$$

Inductive step:

Let n be the predecessor of n' on that shortest path (so $n \rightarrow n' \rightarrow \dots \rightarrow \text{goal}$), i.e., n is $k + 1$ steps away from the goal along a shortest path.

By consistency:

$$h(n) \leq c(n, n') + h(n')$$

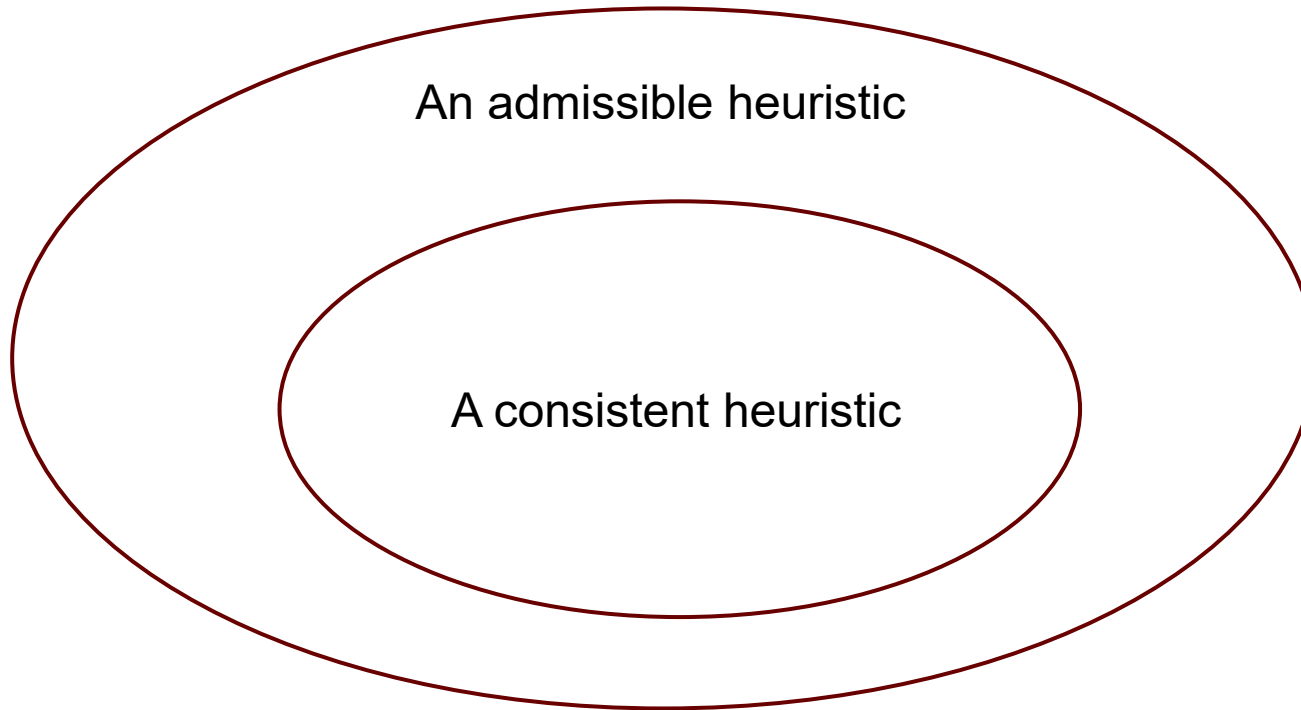
By the *inductive hypothesis*: $h(n') \leq h^*(n')$

So: $h(n) \leq c(n, n') + h^*(n') = h^*(n)$

Therefore, $h(n) \leq h^*(n)$: admissibility holds for n

$h^*(n)$: the actual cost from n to the goal

Q3. Takeaway



A consistent heuristic is always admissible.

However, an admissible heuristic is not always consistent.

Q4.

You are working on implementing A* search in a navigation app that finds the shortest path between two cities on a map. The graph is represented as an adjacency list, edge weights represent driving time, and each city has a precomputed heuristic value $h(n)$, which estimates the remaining travel time to the goal.

Below is a partially written pseudocode for the A* search algorithm. Some parts are missing.

- i. Fill in the blanks (1) – (4) with the correct logic in the pseudocode.
- ii. Explain how the f-score is used to prioritize nodes in the open list.
- iii. The performance of A* heavily depends on the quality of the heuristic $h(n)$. What are the consequences if the heuristic overestimates the actual cost to the goal?

Q4.

```

function A_Star(Graph, start, goal, h):
    for each node n in Graph:
        g[n] ← _____ // (1) cost from start to n
        f[n] ← _____ // (2) estimated total cost (f = g + h)
        prev[n] ← null

    g[start] ← 0
    f[start] ← h[start]
    open ← priority queue ordered by f-value
    Insert(open, start)

    closed ← empty set

    while open is not empty:
        current ← Extract-Min(open)

        if current == goal:
            return Reconstruct-Path(prev, goal)

        Add(closed, current)

        for each neighbor in neighbors of current:
            if neighbor in closed:
                continue

            tentative_g ← g[current] + cost(current, neighbor)

            if tentative_g < g[neighbor]:
                prev[neighbor] ← _____ // (3)
                g[neighbor] ← tentative_g
                f[neighbor] ← _____ // (4)

            if neighbor in open:
                Update-Priority(open, neighbor)
            else:
                Insert(open, neighbor)

    return failure

```

Q4.

Fill in the blanks (1) – (4)

```
function A_Star(Graph, start, goal, h):
```

```
  for each node n in Graph:
```

```
    g[n] ←           
```

```
    f[n] ←           
```

```
    prev[n] ← null
```

```
g[start] ← 0
```

```
f[start] ← h[start]
```

```
open ← priority queue ordered by f-value
```

```
Insert(open, start)
```

```
closed ← empty set
```

Explored set

```
while open is not empty:
```

```
  current ← Extract-Min(open)
```

```
  if current == goal:
```

```
    return Reconstruct-Path(prev, goal)
```

```
  Add(closed, current)
```

```
  for each neighbor in neighbors of current:
```

```
    if neighbor in closed:
```

```
      continue
```

```
    tentative_g ← g[current] + cost(current, neighbor)
```

```
    if tentative_g < g[neighbor]:
```

```
      prev[neighbor] ← current // (3)
```

```
      g[neighbor] ← tentative_g
```

```
      f[neighbor] ← g[neighbor] + h[neighbor] // (4)
```

```
    if neighbor in open:
```

```
      Update-Priority(open, neighbor)
```

```
    else:
```

```
      Insert(open, neighbor)
```

```
return failure
```

- $g(n)$ – the current shortest distance from the start to node n (initial value set to $+\infty$ for all nodes)
- $f(n)$ – the estimated total cost from the start node to the goal through node n , computed as $f(n) = g(n) + h(n)$ (initially set to $+\infty$ for all nodes)

• The distance from the start to itself is 0. • The estimated total cost from the start node to the goal is the heuristic value of the start node.

- Create a priority queue (frontier) ordered by f value.
- Push the start node into the priority queue with its f value.

- Get the node with the smallest f value from the frontier

- Otherwise, mark it as visited, and start to expand the node.

For all neighbouring nodes that are not processed:

If going through current node gives a **shorter distance** to neighbour:

- update the shortest distance and f -value
- push neighbour into the priority queue or update with its new f -value

- If no path is found, return failure.

(ii) Explain how the f-score is used to prioritize nodes in the open list. The f-score is used to estimate the total cost of a path going through a node to the goal.

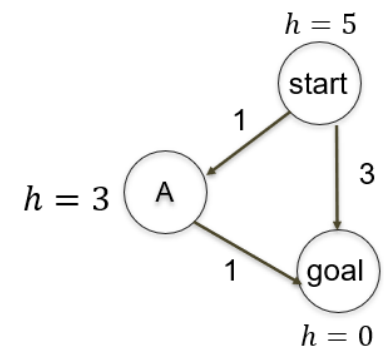
- A* search always selects the node with the lowest f-score from the open list (via priority queue).
- This ensures that paths that appear promising (i.e. with low estimated total cost) are explored first.
- Mathematically:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the known cost from the start to n , and $h(n)$ is the heuristic estimate from n to the goal.

(iii) The performance of A^* heavily depends on the quality of the heuristic $h(n)$. What are the consequences if the heuristic overestimates the actual cost to the goal?

If the heuristic overestimates the actual cost to the goal, A^* is no longer guaranteed to find the optimal path. The algorithm may incorrectly dismiss the true shortest path because the overestimated heuristic makes it seem more expensive than it actually is. As a result, A^* might explore a different path that looks better under the faulty estimate, leading to a valid solution — but not necessarily the shortest one.



Q5. (Optional – For Exploration)

Extend the STRIPS planner from Lecture 4 to model a robot performing a multi-step coffee-making process.

Please refer to TUT3-Q5.py for sample implementation.

Additional references

Python networkx library:

<https://networkx.org/documentation/networkx-0.37/>

Planning: STRIPS:

<https://ai.stanford.edu/~nilsson/OnlinePubs-Nils/PublishedPapers/strips.pdf>

<https://www.geeksforgeeks.org/artificial-intelligence/strips-in-ai/>

